1 Spring in the real world

This chapter covers

- What a framework is
- When to use and when to avoid using frameworks
- What the Spring framework is
- Using Spring in real-world scenarios

The Spring framework (shortly, Spring) is an application framework that is part of the Java ecosystem. An *application framework* is a set of common software functionalities that provides a foundation structure for developing an application. An application framework eases the effort of writing an application by taking out the effort of writing all the program code from scratch.

We use Spring in the development of many kinds of applications nowadays, from large backend solutions to automation testing apps. According to many survey reports on Java technologies (like this one of JRebel from 2020: http://mng.bz/N4V7; or this one from JAXEnter: http://mng.bz/DK9a), Spring is the most used Java framework today.

Spring is popular, and developers have started to use it more often with other JVM languages than Java as well. In the last few years, we observed an impressive

growth of developers using Spring with Kotlin (another appreciated language from the JVM family). In this book, we'll focus on the foundations of Spring, and I'll teach you essential skills for using Spring in real-world examples. To make the subject more comfortable for you and allow you to focus on Spring, we'll use only Java examples. Throughout the book, we'll discuss and apply, with examples, essential skills like connecting to a database, establishing communication between applications, and securing and testing an app.

Before diving into more technical details in the next chapters, let's talk about the Spring framework and where you'll actually use it. Why is Spring so appreciated, and when should you even use it?

In this chapter, we'll focus on what a framework is, referring in particular to the Spring framework. In section 1.1, we discuss the advantages of using a framework. In section 1.2, we discuss the Spring ecosystem with the components you need to learn to get started with Spring. Then I'll take you through possible usages of the Spring framework—in particular, real-world scenarios in section 1.3. In section 1.4, we'll discuss when using frameworks might not be the right approach. You need to understand all these things about the Spring framework before trying to use it. Otherwise, you might try to use a hammer to dig your garden.

Depending on your level, you might feel this chapter difficult. I might introduce some notions that you haven't heard about, and this aspect might disturb you. But don't worry; even if you don't understand some of the things now, they will be clarified later in the book. Sometimes, throughout the book, I'll refer to something said in earlier chapters. I use this approach because learning a framework like Spring doesn't always offer us a linear learning path, and sometimes you need to wait until you get more pieces of the puzzle before you see the complete picture. But in the end, you'll get a clear image, and you'll get the valuable skills you need to develop apps like a professional.

1.1 Why should we use frameworks?

In this section, we discuss frameworks. What are they? How did this concept appear, and why? To be motivated to use something, you need to know how that something brings you value. And that's also the case with Spring. I'll teach you these essential details by sharing the knowledge I gathered from my own experience and by studying and using various frameworks in real-world scenarios, including Spring.

An application framework is a set of functionalities on top of which we build applications. The application framework provides us a broad set of tools and functionalities that you can use to build apps. You don't need to use all the features the framework offers. Depending on the requirements of the app you make, you'll choose the right parts of the framework to use.

Here's an analogy I like for application frameworks. Did you ever buy a piece of furniture from a DIY store like Ikea? Say you buy a wardrobe—you won't get an assembled wardrobe, but the right components you need to build it and a manual on how to assemble your piece of furniture. Now imagine you ordered a wardrobe, but instead

of getting only the right components you need, you get all the possible components you can use to assemble any piece of furniture: a table, a wardrobe, and so on. If you want a wardrobe, you have to find the right parts and assemble them. It's like an application framework. The application framework offers you various pieces of software you need to build your app. You need to know what features to choose and how to assemble them to achieve the right result (figure 1.1).

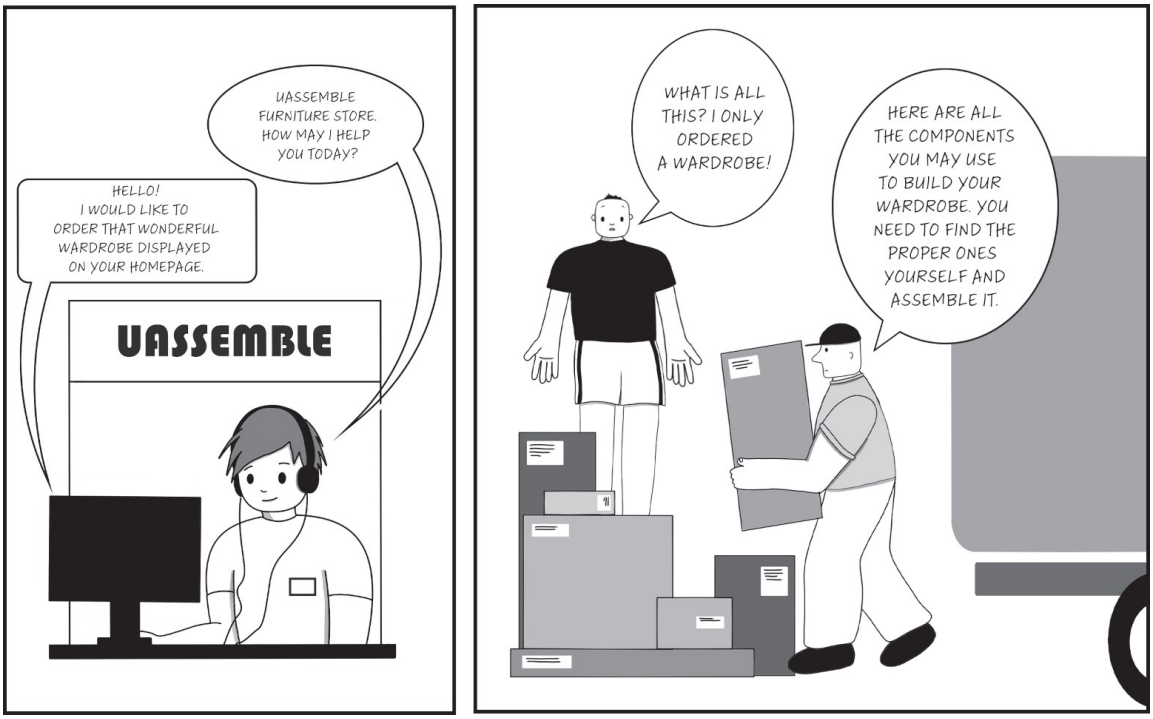

Figure 1.1 David ordered a wardrobe from the UAssemble store. But the store (framework) doesn't deliver to David (the programmer) just the components (software capabilities) he needs to build his new wardrobe (the app). The store ships him all the possible parts he might need to build the wardrobe. It's David's (the programmer's) choice on which components (software capabilities) are right and how to assemble them to get the right result (the application).

The idea of a framework isn't new. Throughout the history of software development, programmers observed they could reuse parts of code they'd written in multiple applications. Initially, when not so many applications were implemented, each application was unique and developed from scratch using a specific programming language. When the software development domain extended, and more and more applications started to be published on the market, it became easier to observe that many of these apps had similar requirements. Let's name a few of them:

- Logging error, warning, and info messages happen in every app.
- Most applications use transactions to process data changes. Transactions represent an important mechanism that takes care of data consistency. We'll discuss this subject in detail in chapter 13.

- Most applications use protection mechanisms against the same common vulnerabilities.
- Most applications use similar ways to communicate with each other.
- Most applications use similar mechanisms to improve their performance, like caching or data compression.

And the list continues. It turns out that the business logic code implemented in an app is significantly smaller than the wheels and belts that make the engine of the application (also often referred to as "the plumbing").

When I say "business logic code," I refer to the code that implements the business requirements of the application. This code is what implements the user's expectations in an application. For example, "clicking on a specific link will generate an invoice" is something users expect to happen. Some code of the application you develop implements this functionality, and this part of code is what developers call the business logic code. However, any app takes care of several more aspects: security, logging, data consistency, and so on (figure 1.2).

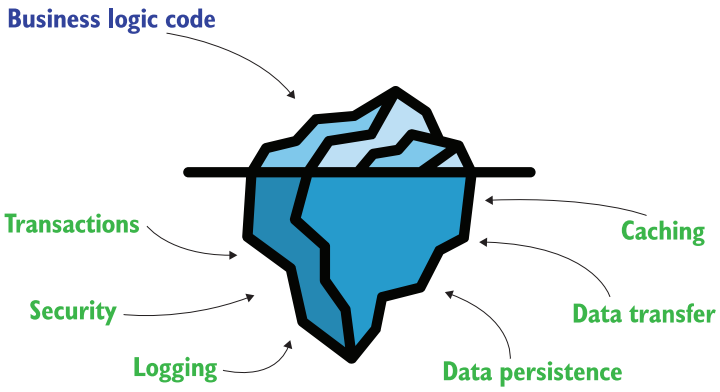

Figure 1.2 The user's perspective is like an iceberg. The users mainly observe the results of the business logic code, but this is only a small part of what builds the app's complete functionality. Like an iceberg has most of it underwater, in an enterprise app we don't see most of the code because it's provided by dependencies.

Moreover, the business logic code is what makes an application different from another from the functionality point of view. If you take two different apps, say a ridesharing system and a social networking app, they have different use cases.

> **NOTE** A *use case* represents the reason a person uses the app. For example, in a ridesharing app, a use case is "requesting a car." For an app managing food delivery, a use case is "ordering a pizza."

You take different actions, but they both need data storing, data transfer, logging, security configurations, probably caching, and so on. Various applications can reuse these nonbusiness implementations. Is it then efficient to rewrite the same functionalities every time? Of course not:

- You spare a lot of time and money by reusing something rather than developing it yourself.
- An existing implementation that many apps already use has fewer chances to introduce bugs, as others have tested it.
- You benefit from the advice of a community because you now have a lot of developers understanding the same functionality. If you had implemented your own code, only a few people would know it.

A story of transition

One of the first applications I worked on was a huge system developed in Java. This system was composed of multiple applications designed around an old-fashioned architecture server, all of them written from scratch using Java SE. The development of this application started with the language about 25 years ago. This was the main reason for its shape. And almost no one could have imagined how big it would become. At that time, more advanced concepts of system architectures didn't exist, and things in general worked differently from the individual systems due to the slow internet connection.

But time passed, and years later, the app was more like a big ball of mud. For valid reasons I won't cover here, the team decided they had to go to a modern architecture. This change implied first cleaning up the code, and one of the main steps was using a framework. We decided to go with Spring. At that time, we had as an alternative Java EE (now named Jakarta EE), but most members of the team considered it's better to go with Spring, which offered a lighter alternative that was easier to implement and that we also considered easier to maintain.

The transition wasn't an easy one. Together with a few colleagues, experts in their domain and knowledgeable about the application itself, we invested a lot of effort into this transformation.

The result was amazing! We removed over 40% of the lines of code. This transition was the first moment I understood how significant the impact of using a framework could be.

NOTE Choosing and using a framework is linked to the design and architecture of an application. You'll find it useful to learn more about these subjects along with learning the Spring framework. In appendix A, you'll find a discussion about software architectures with excellent resources if you'd like to go into details.

1.2 The Spring ecosystem

In this section, we will discuss Spring and related projects like Spring Boot or Spring Data. You'll learn all about these in this book, and the links among them. In real-world scenarios, it's common to use different frameworks together, where each framework is designed to help you implement a specific part of the app faster.

We refer to Spring as a framework, but it is much more complex. Spring is an ecosystem of frameworks. Usually, when developers refer to the Spring framework, they refer to a part of the software capabilities that include the following:

1. *Spring Core*—One of the fundamental parts of Spring that includes foundational capabilities. One of these features is the Spring context. As you'll learn in detail in chapter 2, the Spring context is a fundamental capability of the Spring framework that enables Spring to manage instances of your app. Also, as part of Spring Core, you find the Spring aspects functionality. Aspects help Spring intercept and manipulate methods you define in your app. We discuss more details of the aspects in chapter 6. The Spring Expression Language (SpEL) is another capability you'll find as part of Spring Core, which enables you to describe configurations for Spring using a specific language. All of these are new notions, and I don't expect you to know them yet. But soon you'll understand that Spring Core holds the mechanisms Spring uses to integrate into your app.
2. *Spring model-view-controller (MVC)*—The part of the Spring framework that enables you to develop web applications that serve HTTP requests. We'll use Spring MVC starting in chapter 7.
3. *Spring Data Access*—Also one of the fundamental parts of Spring. It provides basic tools you can use to connect to SQL databases to implement the persistence layer of your app. We'll use Spring Data Access starting in chapter 13.
4. *Spring testing*—The part holding the tools you need to write tests for your Spring application. We'll discuss this subject in chapter 15.

You can initially imagine the Spring framework as a solar system, where Spring Core represents the star in the middle, which holds all the framework together (figure 1.3).

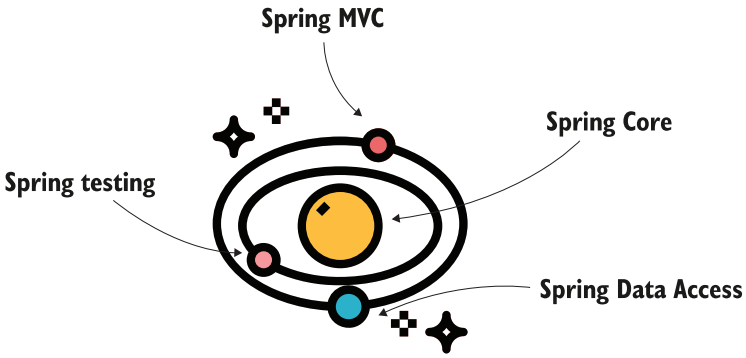

Figure 1.3 You can imagine the Spring framework as a solar system with the Spring Core in the center. The software capabilities are planets around Spring Core kept close to it by its gravitational field.

1.2.1 Discovering Spring Core: The foundation of Spring

Spring Core is the part of the Spring framework that provides the foundational mechanisms to integrate into apps. Spring works based on the principle *inversion of control (*IoC*)*. When using this principle, instead of allowing the app to control the execution, we give control to some other piece of software—in our case, the Spring framework. Through configurations, we instruct the framework on how to manage the code we

write, which defines the logic of the app. Here's where the "inversion" in IoC comes from: you don't let the app control the execution by its own code and use dependencies. Instead, we allow the framework (the dependency) to control the app and its code (figure 1.4).

NOTE In this context the term "controls" refers to actions like "creating an instance" or "calling a method." A framework can create objects of the classes you define in your app. Based on the configurations that you write, Spring intercepts the method to augment it with various features. For example, Spring can intercept a specific method to log any error that might appear during the method's execution.

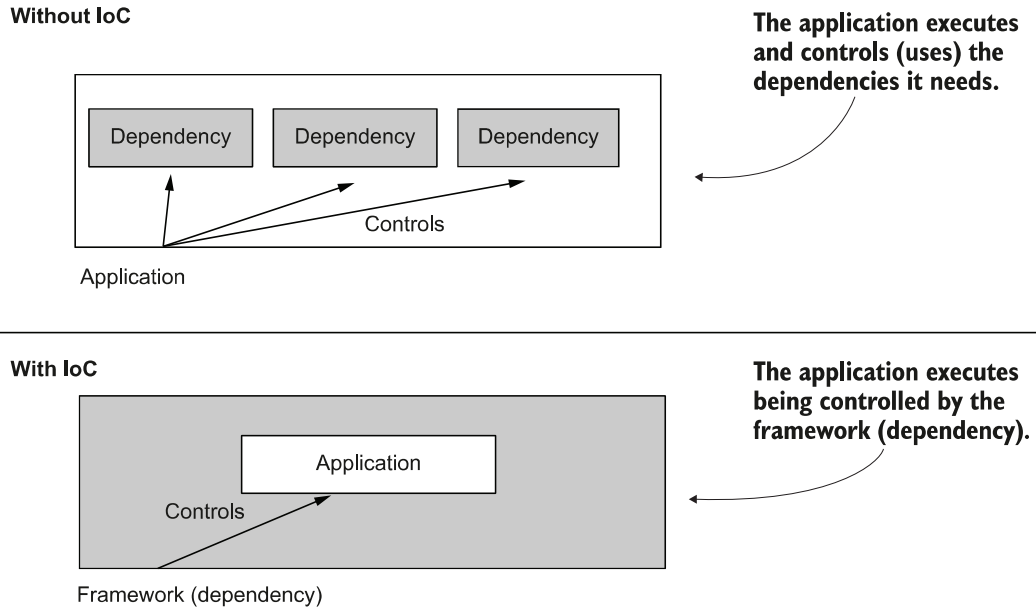

Figure 1.4 Inversion of control. Instead of executing its own code, which makes use of several other dependencies, in case of an IoC scenario, the app execution is controlled by the dependency. The Spring framework controls an app during its execution. Therefore, it implements an IoC scenario of execution.

You will start learning Spring with Spring Core by discussing the Spring IoC functionality in chapters 2 through 5. The IoC container glues Spring components and components of your application to the framework together. Using the IoC container, to which you often refer as the Spring context, you make certain objects known to Spring, which enables the framework to use them in the way you configured.

In chapter 6, we'll continue our discussion with Spring aspect-oriented programming (AOP). Spring can control instances added to its IoC container, and one of the things it can do is intercept methods that represent the behavior of these instances. This capability is called *aspecting* the method. Spring AOP is one of the most common ways the framework interacts with what your app does. This trait makes Spring AOP part of the essentials as well. Part of the Spring Core, we also find resource management,

internationalization (i18n), type conversion, and SpEL. We'll encounter aspects of these features in examples throughout the book.

1.2.2 Using Spring Data Access feature to implement the app's persistence

For most applications, it's critical to persist part of the data they process. Working with databases is a fundamental subject, and in Spring, it's the Data Access module that you'll use to take care of data persistence in many cases. The Spring Data Access includes using JDBC, integrating with *object-relational mapping* (ORM) frameworks like Hibernate (don't worry if you don't yet know what an ORM framework is or haven't heard about Hibernate; we'll discuss these aspects later in the book), and managing transactions. In chapters 12 through 14, we'll cover everything needed to get you started with Spring Data Access.

1.2.3 The Spring MVC capabilities for developing web apps

The most common applications developed with Spring are web apps, and within the Spring ecosystem, you'll find a large set of tools that enables you to write web applications and web services in different fashions. You can use the Spring MVC to develop apps using a standard servlet fashion, which is common in a vast number of applications today. In chapter 7, we'll go into more detail on using the Spring MVC.

1.2.4 The Spring testing feature

The Spring testing module offers us a large set of tools that we'll use to write unit and integration tests. There have been many pages written about the testing topic, but we'll discuss everything that is essential to get you started with Spring testing in chapter 15. I'll also refer to some valuable resources you need to read to get all the details of this topic. My rule of thumb is that you're not a mature developer if you don't understand testing, so this topic is one you should care about.

1.2.5 Projects from the Spring ecosystem

The Spring ecosystem is so much more than just the capabilities discussed earlier in this section. It includes a big collection of other frameworks that integrate well and form a larger universe. Here we have projects like Spring Data, Spring Security, Spring Cloud, Spring Batch, Spring Boot, and so on. When you develop an app, you can use more of these projects together. For example, you can build an app using all of Spring Boot, Spring Security, and Spring Data. In the next few chapters, we'll work on smaller projects that make use of various projects of the Spring ecosystem. When I say project, I refer to a part of the Spring ecosystem that is independently developed. Each of these projects has a separate team that works on extending its capabilities. Also, each project is separately described and has its own reference on the Spring official website: https://spring.io/projects.

Out of this vast universe created by Spring, we'll also refer to Spring Data and Spring Boot. These projects are often encountered in apps, so it's important to get to know them from the beginning.

Extending the persistence capabilities with Spring Data

The Spring Data project implements a part of the Spring ecosystem that enables you to easily connect to databases and use the persistence layer with a minimum number of lines of code written. The project refers to both SQL and NoSQL technologies and creates a high-level layer, which simplifies the way you work with data persistence.

> **NOTE** We have Spring Data Access, which is a module of Spring Core, and we also have an independent project in the Spring ecosystem named Spring Data. Spring Data Access contains fundamental data access implementations like the transaction mechanism and JDBC tools. Spring Data enhances access to databases and offers a broader set of tools, which makes development more accessible and enables your app to connect to different kinds of data sources. We'll discuss this subject in chapter 14.

Spring Boot

Spring Boot is a project part of the Spring ecosystem that introduces the concept of "convention over configuration." The main idea of this concept is that instead of setting up all the configurations of a framework yourself, Spring Boot offers you a default configuration that you can customize as needed. The result, in general, is that you write less code because you follow known conventions and your app differs from others in few or small ways. So instead of writing all the configurations for each and every app, it's more efficient to start with a default configuration and only change what's different from the convention. We'll discuss more about Spring Boot starting in chapter 7.

The Spring ecosystem is vast and contains many projects. Some of them you encounter more often than others, and some you may not use at all if you're building an application without a particular need. In this book, we refer only to the projects that are essential for you to get started: Spring Core, Spring Data, and Spring Boot. You can find a full list of projects that are part of the Spring ecosystem on the official Spring website: https://spring.io/projects/.

Alternatives for using Spring

We can't really discuss alternatives to Spring because someone could misunderstand them as alternatives to the entire ecosystem. But for many of the individual components and projects that create the Spring ecosystem, you can find other options like other open source or commercial frameworks or libraries.

For example, let's take the Spring IoC container. Years ago, the Java EE specification was a solution very much appreciated by the developers. With a slightly different philosophy, Java EE (which in 2017 was open sourced and remade in Jakarta EE, https://jakarta.ee/) offered specifications like Context and Dependency Injection (CDI) or Enterprise Java Beans (EJB). You could use CDI or EJB to manage a context of object instances and implement aspects (named "interceptors" in the EE terminology). Also, throughout history, Google Guice (https://github.com/google/guice) was an appreciated framework for the management of object instances in a container.

(continued)

For some of the projects taken individually, you could find one or more alternatives. For example, you could choose to use Apache Shiro (https://shiro.apache.org/) instead of Spring Security. Or you could decide to implement your web app using the Play framework (https://www.playframework.com/) instead of Spring MVC and Spring-related technologies.

A more recent project that looks promising is Red Hat Quarkus. Quarkus is designed for cloud native implementations and becomes more and more mature with rapid steps. I wouldn't be surprised to see it as one of the lead projects in developing enterprise apps in the Java ecosystem in the future (https://quarkus.io/).

My advice for you is to always take into consideration your alternatives. In software development, you need to be open-minded and never trust one solution as being "the one." You'll always find scenarios in which a specific technology works better than another.

1.3 Spring in real-world scenarios

Now that you have an overview of Spring, you're aware of when and why you should use a framework. In this section, I'll give you some application scenarios in which using the Spring framework might be an excellent fit. Too often, I've seen developers only refer to backend applications for using a framework like Spring. I've even seen a trend of restricting, even more, the scenario to backend web applications. While it's true that in plenty of cases we see Spring used in this way, it's important to remember that the framework isn't limited to this scenario. I've seen teams successfully using Spring in different kinds of applications, such as the development of an automation testing app or even in standalone desktop scenarios.

I'll further describe to you some common real-world scenarios in which I've seen Spring used successfully. These are not the only possible scenarios, and Spring might not work all the time in these cases. Remember what we discussed in section 1.2: a framework is not always a good choice. But these are common cases in which generally Spring is a good fit:

1. The development of a backend app
2. The development of an automation testing framework
3. The development of a desktop app
4. The development of a mobile app

1.3.1 Using Spring in the development of a backend app

A backend application is the part of a system that executes on the server side and has the responsibility of managing data and serving client applications' requests. The users access functionalities by using the client apps directly. Further, the client apps make requests to the backend app to work with the users' data. The backend app